

Java Lab Manual

Object-Oriented Programming & Basics

Covering Classes, Inheritance, Abstraction, and Control Structures.

1. Classes and Objects

Question

Write a Java program to create a book class with attributes for book ID, name, author, and price. Create multiple book objects and display their details.

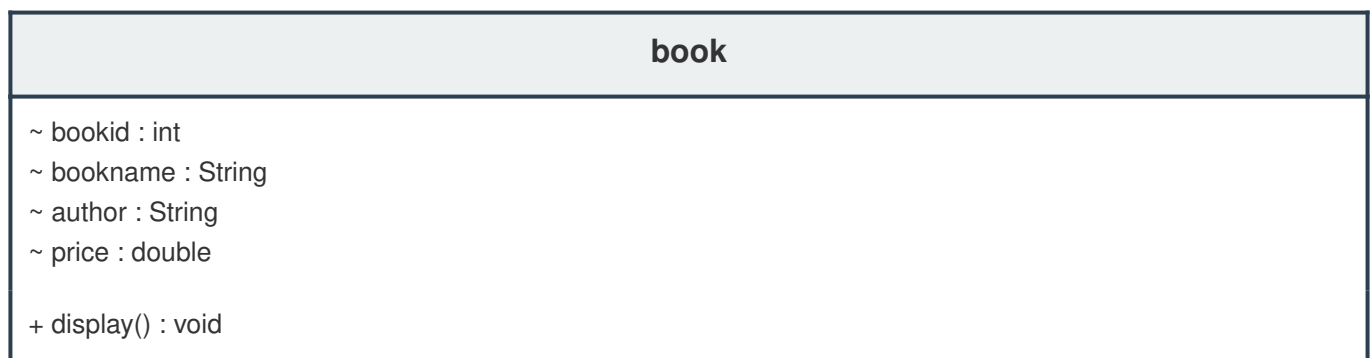
Step-by-Step Process

1. Define a class named book.
2. Declare instance variables: bookid (int), bookname (String), author (String), and price (double).
3. Create a method display() to print the details of the book.
4. In the main class Task2, instantiate two book objects (b and l).
5. Assign values to the instance variables for both objects.
6. Call the display() method for each object to print their details.

Algorithm

1. **START**
2. Create class book with properties and a display method.
3. Create the main driver class Task2.
4. Initialize object b of type book.
5. Set b.bookid = 244, b.bookname = "java learns", b.author = "james gosling", b.price = 250.00.
6. Initialize object l of type book.
7. Set l.bookid = 28, l.bookname = "learn", l.author = "suresh", l.price = 45.50.
8. Call b.display() and l.display().
9. **STOP**

UML Diagram



Code Implementation

```
class book {
    int bookid;
    String bookname;
    String author;
    double price;

    void display() {
        System.out.println("book id name " + bookid);
        System.out.println("bookname is " + bookname);
        System.out.println("book author is " + author);
        System.out.println("the price of book is " + price);
    }
}

public class Task2 {
    public static void main(String[] args) {
        book b = new book();
        b.bookid = 244;
        b.bookname = "java learns";
        b.author = "james gosling";
        b.price = 250.00;

        book l = new book();
        l.bookid = 28;
        l.bookname = "learn";
        l.author = "suresh";
        l.price = 45.50;

        b.display();
        l.display();
    }
}
```

Sample Output

```
book id name 244
bookname is java learns
book author is james gosling
the price of book is 250.0
book id name 28
bookname is learn
book author is suresh
the price of book is 45.5
```

2. Inheritance

Question

Write a Java program to demonstrate inheritance. Create a parent class person and derive two child classes: student and faculty. Display their respective details.

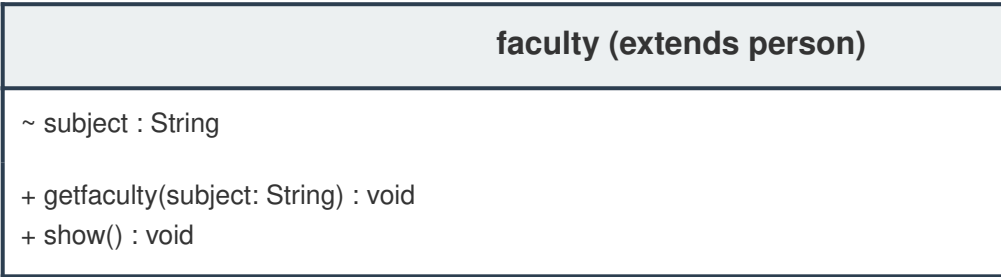
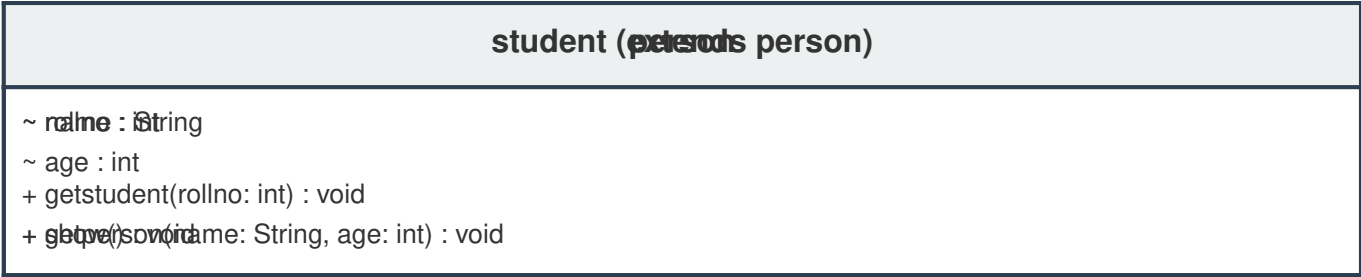
Step-by-Step Process

1. Create a base class person with attributes name and age, and a method `getperson()`.
2. Create a child class student extending person with a specific attribute rollno.
3. Add methods `getstudent()` and `show()` in the student class to assign and display data.
4. Create another child class faculty extending person with attribute subject.
5. Add methods `getfaculty()` and `show()` in the faculty class.
6. In the main method, instantiate student and faculty objects, populate them, and invoke their `show()` methods.

Algorithm

1. **START**
2. Define base class person.
3. Define derived classes student and faculty inheriting from person.
4. In main class Task21, create object s of student.
5. Call `s.getperson("hemanth", 19)` and `s.getstudent(244)`.
6. Create object f of faculty.
7. Call `f.getperson("srinu", 35)` and `f.getfaculty("programming")`.
8. Call `s.show()` and `f.show()`.
9. **STOP**

UML Diagram



Code Implementation

```
class person {
    String name;
    int age;
    void getperson(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

class student extends person {
    int rollno;
    void getstudent(int rollno) {
        this.rollno = rollno;
    }
    void show() {
        System.out.println("---student details---");
        System.out.println("name is " + name);
        System.out.println("age is " + age);
        System.out.println("rollno is " + rollno);
    }
}

class faculty extends person {
    String subject;
    void getfaculty(String subject) {
        this.subject = subject;
    }
    void show() {
        System.out.println("--- faculty details---");
        System.out.println("name of faculty is " + name);
        System.out.println("age of " + age);
        System.out.println("subject is " + subject);
    }
}

public class Task21 {
    public static void main(String[] args) {
        student s = new student();
        s.getperson("hemanth", 19);
        s.getstudent(244);

        faculty f = new faculty();
        f.getperson("srinu", 35);
        f.getfaculty("programming");

        s.show();
        f.show();
    }
}
```

Sample Output

```
---student details---  
name is hemanth  
age is 19  
rollno is 244  
--- faculty details---  
name of faculty is srinu  
age of 35  
subject is programming
```

3. Abstraction and Interfaces

Question

Write a Java program to implement abstraction using an abstract class shape and an interface printable.

Step-by-Step Process

1. Create an abstract class shape with an abstract method draw().
2. Create classes circle and rectangle that extend shape and override the draw() method.
3. Define an interface printable with a method signature ward().
4. Create a class report that implements printable and defines the ward() method.
5. In the main method, create objects for report, circle, and rectangle, and call their respective methods.

Algorithm

1. **START**
2. Define abstract class shape and interface printable.
3. Implement classes circle, rectangle (extending shape) and report (implementing printable).
4. In Task23 main method, initialize report `r = new report()`.
5. Call `r.ward()`.
6. Initialize circle `a = new circle()` and call `a.draw()`.
7. Initialize rectangle `t = new rectangle()` and call `t.draw()`.
8. **STOP**

UML Diagram



Code Implementation

```
abstract class shape {
    abstract void draw();
}

class circle extends shape {
    @Override
    public void draw() {
        System.out.println("the drawing is circle");
    }
}

class rectangle extends shape {
    @Override
    public void draw() {
        System.out.println("th edrawing is rectangle");
    }
}

interface printable {
    void ward();
}

class report implements printable {
    @Override
    public void ward() {
        System.out.println("the interface is impenents");
    }
}

public class Task23 {
    public static void main(String[] args) {
        report r = new report();
        r.ward();
        circle a = new circle();
        a.draw();
        rectangle t = new rectangle();
        t.draw();
    }
}
```

Sample Output

```
the interface is impenents
the drawing is circle
th edrawing is rectangle
```

4. Conditional Statements

Question

Write a Java program to check whether a given integer is even or odd.

Step-by-Step Process

1. Import the Scanner class.
2. Prompt the user to enter an integer.
3. Read the integer using `nextInt()`.
4. Use an `if-else` block to check if the number modulo 2 equals 0.
5. Print whether the number is even or odd.

Algorithm

1. **START**
2. Initialize a Scanner object.
3. Read integer n .
4. If $n \% 2 == 0$, print "the number is even".
5. Else, print "the number is odd".
6. **STOP**

Code Implementation

```
import java.util.Scanner;

public class ConditionalState {
    public static void main(String[] args) {
        Scanner eo = new Scanner(System.in);
        int n;
        System.out.println("enter the number");
        n = eo.nextInt();
        if(n % 2 == 0){
            System.out.println("the number is even");
        }
        else {
            System.out.println("the number is odd");
        }
    }
}
```

Sample Output

```
enter the number  
10  
the number is even
```

5. Looping Statements

Question

Write a Java program to generate and print the multiplication table of a given number.

Step-by-Step Process

1. Import the Scanner utility.
2. Take an integer input n from the user.
3. Use a for loop starting from $i = 1$ to $i = 10$.
4. Calculate $n * i$ and print the formatted string.

Algorithm

1. **START**
2. Read integer n .
3. Initialize loop counter $i = 1$.
4. If $i \leq 10$, calculate product = $n * i$.
5. Print the result.
6. Increment i and repeat step 4.
7. **STOP**

Code Implementation

```
import java.util.Scanner;

public class LoopState {
    public static void main(String[] args) {
        Scanner m = new Scanner(System.in);
        int n;
        System.out.println("enter the number");
        n = m.nextInt();
        System.out.println("the multipilcation table of given number");

        for(int i = 1; i <= 10; i++){
            System.out.println(n + "x" + i + " = " + (n*i));
        }
    }
}
```

Sample Output

```
enter the number
5
the multipilcation table of given number
5x1 = 5
5x2 = 10
5x3 = 15
5x4 = 20
5x5 = 25
5x6 = 30
5x7 = 35
5x8 = 40
5x9 = 45
5x10 = 50
```